University of Applied Sciences
Fakultät für Informatik und Wirtschaftsinformatik

MoveLink

Visualisierung von Echtzeit-Bewegungsdaten

Projektbericht

Software-Architekturen Labor

Sommersemester 2026

Betreuer: Prof. Dr. rer. nat. Sinz

Team: Erlind Sejdiu
Luca Schöneberg
Devin Uyan

Abgabe: Ende Sommersemester 2026

Version 2.0 – Stand: 16. April 2026

Inhaltsverzeichnis

1 Einleitung **1**

1.1 Motivation 1

1.2 Zielsetzung 1

1.3 Abgrenzung 1

2 Grundlagen **2**

2.1 Problemstellung 2

2.2 Stakeholder 2

2.3 Technische Grundlagen 2

2.3.1 Bluetooth Low Energy (BLE) und GATT 2

2.3.2 IMU-Sensoren 3

2.3.3 Echtzeit-Kommunikation via WebSockets 3

3 Use Cases und Anforderungen **3**

3.1 Use Cases 3

3.2 Systemanforderungen 4

4 Systemarchitektur **5**

4.1 Gewählte Architektur: Containerisierte Monolith-Architektur 5

4.2 Diskussion von Alternativen 5

5 Technologieauswahl **6**

6 Umsetzung und Implementierung **7**

6.1 Embedded-Schicht – XIAO nRF52840 (AP 3) 7

6.2 Backend – API-Gateway und Containerisierung (AP 2) 7

6.3 Frontend – Mobile Applikation (AP 1) 7

7 Qualitätssicherung und Tests **8**

8 Fazit **8**

Literaturverzeichnis **8**

1 Einleitung

1.1 Motivation

Die Qualität der Bewegungsausführung ist im Krafttraining ein zentraler Faktor für den langfristigen Trainingserfolg und die Vermeidung von Verletzungen. Während professionelle Athleten Zugang zu Coaches und biomechanischer Analyse-Hardware haben, trainieren Breitensportler überwiegend ohne jegliches objektives Feedback. Fehler in der Technik – beispielsweise ein zu stark gerundeter Rücken bei der Kniebeuge – werden dadurch oft über lange Zeit nicht korrigiert und können zu chronischen Beschwerden führen.

Gleichzeitig sind Miniaturisierung und Kostensenkung von Inertial-Measurement-Unit-Sensoren (IMU) in den vergangenen Jahren stark vorangeschritten. Geräte wie der XIAO nRF52840 von Seeed Studio kombinieren einen leistungsfähigen Mikrocontroller mit integrierter Bluetooth-Low-Energy-Schnittstelle (BLE) und ermöglichen damit den Aufbau kostengünstiger, kabelloser Sensorknoten. Diese Entwicklung eröffnet die Möglichkeit, Bewegungsanalyse aus dem Hochleistungssport in den Alltag des Breitensports zu bringen.

1.2 Zielsetzung

Ziel dieses Laborprojekts ist die prototypische Entwicklung des Systems **MoveLink**, das die vollständige Datenkette von der Sensorerfassung bis zur visuellen Darstellung im Frontend abbildet. MoveLink soll als **Proof of Concept (PoC)** demonstrieren, dass es möglich ist, IMU-Rohdaten in Echtzeit drahtlos zu erfassen, serverseitig zu verarbeiten und für den Trainierenden verständlich zu visualisieren.

Das Projekt gliedert sich in drei voneinander abgegrenzte Arbeitspakete, die parallel von den Teammitgliedern bearbeitet werden:

- **AP 1 – Frontend (Devin Uyan):** Entwicklung der mobilen Applikation zur Echtzeit-Visualisierung der Bewegungsdaten und Verwaltung der Sensorverbindung.
- **AP 2 – Backend (Luca Schöneberg):** Implementierung des API-Gateways, der WebSocket-Infrastruktur sowie der Docker-Containerisierung aller Dienste.
- **AP 3 – Embedded (Erlind Sejdiu):** Firmware-Entwicklung für den XIAO nRF52840, Kalibrierung der IMU-Sensoren und Implementierung der BLE-Kommunikation.

1.3 Abgrenzung

Die automatische **Bewegungserkennung und -bewertung via Machine Learning** ist explizit nicht Bestandteil dieses Projekts. Diese Komponente wird in einem separaten Projekt von Erlind Sejdiu behandelt. Im Rahmen von MoveLink werden die Rohdaten des Sensors (Beschleunigung, Winkelgeschwindigkeit) visualisiert und einfache, schwellenwertbasierte Auswertungen implementiert. Eine KI-gestützte Qualitätsbewertung ist als optionale Erweiterung (Kann-Kriterium) vorgesehen.

2 Grundlagen

2.1 Problemstellung

Im Krafttraining ist eine korrekte Bewegungsausführung essenziell für den Trainingserfolg und die Verletzungsprävention. Dennoch fehlt es den Trainierenden in der Praxis meist an unmittelbarem, objektivem Feedback zur Qualität ihrer Ausführung.

Zwar können Bewegungsabläufe durch moderne Hardware-Sensoren erfasst werden, jedoch liegen diese Informationen oft nur als unstrukturierte Rohdaten vor, die für den Endanwender kaum interpretierbar sind. Daraus ergibt sich die Herausforderung, die anfallenden Datenströme zentral zusammenzuführen und visuell nutzergerecht aufzubereiten.

Um diese Lücke zu schließen, wird im Rahmen dieses Laborprojekts die Applikation MoveLink konzipiert und prototypisch entwickelt. Ziel ist es, als zentrale Schnittstelle zu fungieren, die Trainingsdaten von Hardware-Sensoren standardisiert erfasst, speichert und in einer einheitlichen Benutzeroberfläche verständlich darstellt.

2.2 Stakeholder

Stakeholder	Interessen und Erwartungen
Trainierende (primär)	Echtzeit-Feedback zur Bewegungsausführung; einfache Bedienung; verlässliche Daten; minimale Latenz.
Entwickler-Team	Klare Modulabgrenzung; testbare Schnittstellen; reproduzierbare Build-Umgebung.
Hochschule (HKA)	Demonstration softwarearchitektonischer Konzepte; vollständige Dokumentation; Einhaltung des Zeitrahmens.
Dienstleister (zukünftig)	Erweiterbarkeit der Plattform; saubere API-Dokumentation; Containerisierbarkeit für Deployment.

Tabelle 1: Stakeholder-Analyse

2.3 Technische Grundlagen

2.3.1 Bluetooth Low Energy (BLE) und GATT

Bluetooth Low Energy (BLE, auch Bluetooth 4.0+) ist ein Funkstandard, der für energiesparende, drahtlose Kommunikation kurzer Reichweite ($\leq 100\,\mathrm{m}$) optimiert ist. Im Gegensatz zu klassischem Bluetooth ist BLE auf sporadische, kurze Datenübertragungen ausgelegt und eignet sich daher ideal für Sensoranwendungen.

Die Kommunikation in BLE basiert auf dem **Generic Attribute Profile (GATT)**, das ein Client-Server-Modell definiert. Der Sensor fungiert als *GATT-Server* und stellt Messwerte als *Characteristics* in einem *Service* bereit. Die mobile App verbindet sich als *GATT-Client* und abonniert Änderungen via *Notifications*, wodurch der Sensor neue Messwerte aktiv an den Client pushen kann.

2.3.2 IMU-Sensoren

Eine Inertial Measurement Unit (IMU) ist eine elektronische Einheit, die mit Hilfe von Beschleunigungssensoren (Accelerometer) und Drehraten-Sensoren (Gyroskop) die Bewegung und Orientierung eines Objekts im dreidimensionalen Raum misst. Die Ausgabedaten bestehen typischerweise aus:

- **Beschleunigung** in m/s^2 auf den Achsen x, y, z
- **Winkelgeschwindigkeit** in rad/s (Gyroskop) auf den Achsen x, y, z

Im Projekt wird der auf dem XIAO nRF52840 Sense verfügbare **LSM6DS3TR-C** verwendet, ein 6-Achsen-IMU von STMicroelectronics (3-Achsen-Accelerometer + 3-Achsen-Gyroskop).

2.3.3 Echtzeit-Kommunikation via WebSockets

Das WebSocket-Protokoll (RFC 6455) ermöglicht eine bidirektionale, vollduplex-fähige Verbindung zwischen Client und Server über eine einzige TCP-Verbindung. Im Gegensatz zu HTTP-Polling hält WebSocket die Verbindung dauerhaft offen, was Overhead durch wiederholte Verbindungsaufbauten eliminiert und die Latenz erheblich reduziert. Für die Echtzeit-Übertragung von Sensordaten (kontinuierlicher Stream) ist WebSocket daher der bevorzugte Mechanismus.

3 Use Cases und Anforderungen

3.1 Use Cases

UC-1: Echtzeit-Training überwachen

Akteur: Trainierender *Vorbedingung:* Trainingsgerät verbunden

Beschreibung: Als Trainierender möchte ich mein Training in Echtzeit überwachen können, um direkt Feedback zu meiner Ausführung zu erhalten.

1. Trainierender startet eine Trainingseinheit in der App.
2. Sensor erfasst Bewegungsdaten und überträgt sie via BLE an das Backend.
3. Backend leitet die Daten per WebSocket an die App weiter.
4. App visualisiert die Bewegung in Echtzeit ($< 100\,\text{ms}$ Latenz).

UC-2: Trainingsgerät verbinden

Akteur: Trainierender *Vorbedingung:* Trainingsgerät eingeschaltet, Backend läuft

Beschreibung: Als Trainierender möchte ich mein Trainingsgerät mit der App verbinden können, um Trainingsdaten erfassen zu können.

1. App zeigt verfügbare BLE-Sensoren in der Nähe an.
2. Trainierender wählt den gewünschten Sensor aus.
3. Verbindung wird hergestellt und der Verbindungsstatus wird in der App angezeigt.

UC-3: Trainingsdaten einsehen

Akteur: Trainierender *Vorbedingung:* Mindestens eine abgeschlossene Trainingseinheit

Beschreibung: Als Trainierender möchte ich vergangene Trainingseinheiten einsehen können, um meine Fortschritte zu verfolgen.

1. Trainierender navigiert zur Historien-Ansicht in der App.
2. App ruft die gespeicherten Einheiten vom Backend ab und zeigt sie an.

3.2 Systemanforderungen

Funktionale Anforderungen – Muss-Kriterien

- **M1:** Die App muss eine BLE-Verbindung zu Trainingssensoren aufbauen können.
- **M2:** Die App muss Echtzeit-Datenströme empfangen und verarbeiten können.
- **M3:** Die App muss eine GUI zur Live-Visualisierung der Bewegungsdaten bereitstellen.
- **M4:** Die App muss eine Historie zum Anzeigen vergangener Trainingseinheiten anbieten.
- **M5:** Das Backend muss als Docker-Container lokal betreibbar sein.
- **M6:** Das Gesamtsystem muss auf Android, macOS und Windows funktionsfähig sein.

Funktionale Anforderungen – Kann-Kriterien

- **K1:** Die App kann Feedback zur Bewegungsqualität ausgeben (z. B. schwellenwertbasierte Warnung).
- **K2:** Die App kann statistische Auswertungen (z. B. Fortschrittskurven) anzeigen.
- **K3:** Mehrere Sensoren können gleichzeitig verbunden und visualisiert werden.

Nicht-funktionale Anforderungen – Muss-Kriterien

- **NF1 – Latenz:** Die E2E-Latenz vom Sensor bis zur Darstellung muss $\leq$ **100 ms** sein.
- **NF2 – Zuverlässigkeit:** Bei einem Verbindungsabbruch muss die App den Nutzer benachrichtigen und automatisch Reconnect-Versuche starten.
- **NF3 – Usability:** Das Pairing darf maximal zwei Nutzerinteraktionen erfordern.
- **NF4 – Reproduzierbarkeit:** Das Gesamtsystem muss per `docker compose up`, in einer neuen Umgebung startbar sein.

4 Systemarchitektur

4.1 Gewählte Architektur: Containerisierte Monolith-Architektur

Die Architektur konzentriert sich auf die Applikationsschicht und folgt einem dreischichtigen Container-Modell, das modular aufgebaut ist und sich optimal für ein lokales Deployment eignet:

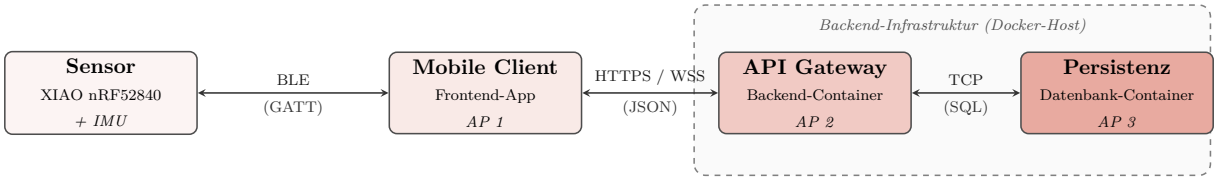

Abbildung 1: Übersicht der Systemarchitektur, Protokolle und Arbeitspakete (AP)

- **Sensor (extern):** Der XIAO nRF52840 mit IMU-Sensor erfasst Bewegungsdaten und sendet sie via BLE/GATT an die App. Er ist bewusst außerhalb des Docker-Hosts angesiedelt, da es sich um externe Hardware handelt.
- **Frontend-App (AP 1):** Stellt die Benutzeroberfläche für den Trainierenden bereit und ist zuständig für BLE-Verbindungsmanagement sowie Echtzeit-Visualisierung.
- **Backend-Container (AP 2):** Isolierter Server als zentrales API-Gateway. Verarbeitet Echtzeit-Datenströme und bietet REST-Schnittstellen für statische Anfragen.
- **Datenbank-Container (AP 3):** Kapselt die Datenhaltung. Speichert Nutzerprofile sowie die für UC-3 benötigten historischen Trainingsdaten persistent.

4.2 Diskussion von Alternativen

Für die Umsetzung des Projekts steht ein Zeitbudget von insgesamt rund 180 Stunden (ca. 60 Stunden pro Person) zur Verfügung. Zur Evaluierung der optimalen Systemarchitektur wurden drei Alternativen untersucht:

Architekturansatz	Vorteile	Nachteile
Client-Server (Microservices) Entkoppelte Frontend-/Backend-Services	• Klare *Separation of Concerns* • Unabhängig skalierbar • Hohe Technologie-Flexibilität	• Hoher Infrastrukturaufwand • Netzwerklatenzen • Hohe Komplexität (PoC)
Client-Server (Monolith) Zentrale Backend-Logik	• Geringe Latenz (In-Memory) • Einfaches Deployment & Testing • Geringer Overhead (PoC)	• Nur vertikale Skalierung • Starke Kopplung bei Wachstum • *Single Point of Failure*
Publisher-Subscriber Ereignisgesteuert (z. B. MQTT)	• Sehr gut skalierbar • Ideal für Daten-Streams (IoT) • Lose gekoppelte Kommunikation	• Broker erforderlich • Hoher Overhead (PoC) • Erschwertes Debugging

Tabelle 2: Vergleich der evaluierten Architekturmuster

Begründung:

Unter Abwägung der Ressourcenrestriktionen (180 Stunden) erweist sich die **monolithische Client-Server-Architektur** als die zielführendste Lösung. Sie minimiert den Infrastruktur-Overhead, erlaubt schnelle Entwicklungszyklen und erfüllt die Echtzeit-Anforderungen durch direkte In-Memory-Aufrufe. Potenzielle Konflikte bei der parallelen Team-Entwicklung werden durch eine saubere interne Modulstruktur vermieden.

5 Technologieauswahl

Die Auswahl der Technologien orientiert sich an den Ressourcenrestriktionen (Zeiteffizienz) sowie der Notwendigkeit, die Arbeitspakete klar voneinander zu entkoppeln.

Schicht	Technologie	Entscheidungsbegründung
Embedded (AP 3)	XIAO nRF52840 C/C++ (Arduino/Zephyr)	Integriertes BLE; kompakter Formfaktor; stromsparend; direkte IMU-Anbindung über I^2C.
Mobile Client (AP 1)	Cross-Platform Framework (React Native / Flutter)	• Single-Codebase für iOS und Android. • Etablierte BLE-Bibliotheken (GATT-Support). • Hohe Rendering-Performance für Live-Charts.
API & Gateway (AP 2)	Node.js (TypeScript) oder Python (FastAPI)	• Non-Blocking I/O für parallele Live-Streams. • Native WebSocket- und REST-Unterstützung.
Persistenz	PostgreSQL	ACID-konforme Speicherung; JSONB-Support für schemaloser Sensordaten (Zeitreihen).
Echtzeit-kommunikation	WebSockets (WSS)	Bidirektionaler Vollduplex-Kanal; Latenz < 100 ms; kein HTTP-Polling-Overhead.
Infrastruktur	Docker & Docker Compose	Reproduzierbarer lokaler Betrieb aller Dienste mit einem Befehl; kein Cloud-Hosting nötig.

Tabelle 3: Übersicht und Begründung des gewählten Technologie-Stacks

6 Umsetzung und Implementierung

6.1 Embedded-Schicht – XIAO nRF52840 (AP 3)

TODO – Wird nach Abschluss der Implementierung ergänzt

(AP 3 – Erlind Sejdiu)

- Beschreibung der Firmware-Architektur (Arduino/Zephyr RTOS)
- IMU-Initialisierung und Kalibrierung (LSM6DS3TR-C über I^2C)
- Abtastrate und Datenformat der Sensorwerte (Accelerometer, Gyroskop)
- BLE-Service-Definitionen (GATT-Service UUID, Characteristic-Layout)
- Implementierung der BLE-Notifications / Advertising
- Energieverwaltung und Verbindungsparameter
- Code-Snippets der Kernfunktionalität

6.2 Backend – API-Gateway und Containerisierung (AP 2)

TODO – Wird nach Abschluss der Implementierung ergänzt

(AP 2 – Luca Schöneberg)

- Projektstruktur und gewähltes Framework (Node.js/FastAPI)
- REST-API-Endpunkte (Routen, Request-/Response-Schemata)
- WebSocket-Handler: Verbindungsmanagement und Broadcast-Logik
- Datenbankschema (Tabellen: Nutzer, Sessions, Sensorwerte)
- Docker-Compose-Konfiguration (Services, Netzwerke, Volumes)
- Umgebungsvariablen und Konfigurationsmanagement
- Sequenzdiagramm des Datenflusses (BLE-Eingang bis WS-Ausgang)

6.3 Frontend – Mobile Applikation (AP 1)

TODO – Wird nach Abschluss der Implementierung ergänzt

(AP 1 – Devin Uyan)

- Projektstruktur und Komponentenarchitektur (z. B. React Native Screens)
- BLE-Verbindungsmanagement: Scan, Pairing, GATT-Notification-Handling
- WebSocket-Client-Implementierung und Zustandsmanagement
- Echtzeit-Visualisierungskomponente (Bibliothek, Darstellungsform, Achsen)
- Historien-Ansicht: Abruf und Darstellung vergangener Sessions
- UI/UX-Entscheidungen (Navigation, Farbgebung, Feedbackelemente)
- Screenshots / Screen-Recordings der fertigen App

7 Qualitätssicherung und Tests

TODO – Wird nach Abschluss der Implementierung ergänzt

Geplante Inhalte dieses Abschnitts:

- **Teststrategie:** Übersicht der eingesetzten Testmethoden (Unit-, Integrations-, Systemtest)
- **Backend-Tests:** Unit-Tests der REST-Endpunkte; Integrationstests des WebSocket-Handlers
- **Frontend-Tests:** Komponenten-Tests; manuelle UI-Tests auf Android und iOS
- **End-to-End-Test:** Messung der tatsächlichen End-to-End-Latenz (Sensor → App)
- **Latenzauswertung:** Vergleich gemessene Latenz vs. Anforderung NF1 ($< 100\,\text{ms}$)
- **Bekannte Einschränkungen:** Abweichungen vom ursprünglichen Pflichtenheft

8 Fazit

TODO – Wird am Projektende verfasst

Geplante Inhalte dieses Abschnitts:

- **Zusammenfassung:** Was wurde umgesetzt? Wurden die Ziele aus Abschnitt 1.2 erreicht?
- **Erfüllte Muss- und Kann-Kriterien:** Rückblick auf den Anforderungskatalog (Abschnitt 3.2)
- **Erfahrungen:** Team-Reflexion zu Technologiewahl, Architekturentscheidungen und Arbeitsteilung
- **Ausblick:** Mögliche nächste Schritte (ML-Integration, Produktivbetrieb, Multi-Sensor-Unterstützung)

Literaturverzeichnis

Literatur

[1] Bluetooth SIG, *Bluetooth Core Specification 5.4*, 2023.
`https://www.bluetooth.com/specifications/specs/core54-html/`

[2] Seeed Studio, *XIAO nRF52840 (Sense) Wiki*, 2023.
`https://wiki.seeedstudio.com/XIAO_BLE/`

[3] STMicroelectronics, *LSM6DS3TR-C Datasheet*, 2019.
`https://www.st.com/en/mems-and-sensors/lsm6ds3tr-c.html`

[4] I. Fette, A. Melnikov, *RFC 6455: The WebSocket Protocol*, IETF, 2011.
`https://datatracker.ietf.org/doc/html/rfc6455`

[5] Docker Inc., *Docker Compose Documentation*, 2024.
`https://docs.docker.com/compose/`